



CSE432: Automata and Computability Term Project

Ahmed Mohamed Elsayed Tabbash	20P1076
Ahmed Abdallah Nofull	22P0255
Mostafa Ayman Mostafa	20p9883
Youssef Tarek Kamal Zaki Mohamed	22P0236

Submitted to:
Prof. Gamal A. Ebrahim
TA Sally E. Shaker

Contents

Definition	3
Advantages & Disadvantages.....	4
Advantages.....	4
Disadvantages	5
Problems it Solves Or Overcome	5
Shortages and Drawbacks	6
Extra Knowledge	6
Deterministic vs. nondeterministic PDA	6
Computation	6
Accepting and Rejecting Computations.....	7
Nondeterministic spontaneous transitions.....	7
Nondeterministic Computations	8
Example.....	8
Threads of Configuration.....	9
Represent Some Computations	9
Turing Machine	10
Definition.....	10
Implementations.....	11
Advantages and disadvantages.....	11
Disadvantages	11
Extra Knowledge	14
History	14
Computing with Turing Machines	15
Examples on CFG to PDA:	18
References	22

Definition :

An automaton that makes use of a stack is known as a pushdown automaton (PDA) in the theory of computation, a subfield of theoretical computer science.

Pushdown automata are used in theories about what can be computed by machines. They are more capable than finite-state machines but less capable than Turing machines

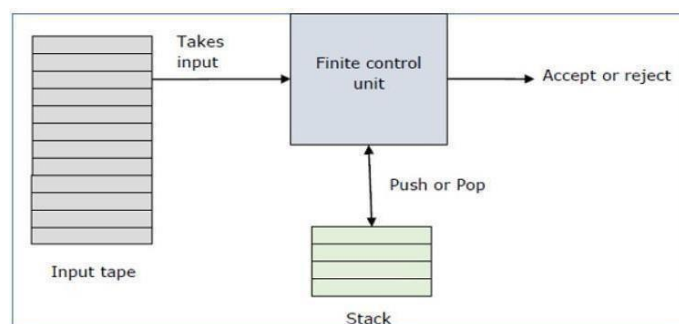
Deterministic pushdown automata can recognize all deterministic context-free languages while nondeterministic ones can recognize all context-free languages, with the former often used in parser design.

The phrase "pushdown" describes how the stack can be thought of as being "pushed down" because operations are only ever performed on the top element.

A pushdown automata has three components

- An input tape
- A Finite control unit
- A stack with infinite size

It may or may not read an input symbol, but it has to read the top of the stack in every transition.



A PDA can formally described as a 7 tuple $(Q, \Sigma, S, \delta, q_0, I, F)$

- **Q** is the finite number of states
- Σ is input alphabet
- **S** is stack symbols
- δ is the transition function: $Q \times (\Sigma \cup \{\epsilon\}) \times S \times Q \times S^*$
- **q₀** is the initial state ($q_0 \in Q$)
- **I** is the initial stack top symbol ($I \in S$)
- **F** is a set of accepting states ($F \subseteq Q$)

Advantages & Disadvantages:

Advantages :

- Pushdown Automata (PDA) can verify regular language as well as context free language.
- A stack is an additional component of a PDA. Additional memory is provided by the stack.
- Languages can be accepted by the PDA in two different ways: by the empty stack and by the final state.
- There is a known algorithm available to construct PDA from a grammar (if the grammar is context free).
- For all regular languages, we can construct a deterministic PDA.

Disadvantages:

- All nondeterministic finite automata can be converted to their corresponding deterministic automata, however all nondeterministic PDAs cannot be converted to deterministic PDAs.
- There is no known algorithm to determine if there exists a deterministic PDA for a grammar.
- The expressive power of non-deterministic PDA is more than the expressive power of deterministic PDA. It means there exists a context free grammar for which we cannot construct deterministic PDA.

For example, we cannot construct deterministic PDA for $L = \{W \mid W \text{ is Palindrome}\}$

- It is undecidable if a PDA recognizes, Σ^* set of all strings over input characters.
- PDA is unable to understand context-sensitive and permissive language.

Problems it Solves Or Overcome:

Pushdown automata used to solve many problems such that it's used for solving Tower of Hanoi Problem which considered of high complexity. Also, FAs can recognize all regular languages, but sometimes PDAs can do it "better".

Sometimes the stack can actually help. For example, PDAs are very good at counting things.

Consider the problem of designing an automaton to recognize strings with k occurrences of 101, where k is some constant.

Also pushdown automata can recognize context free languages. The classic example of a context-free language which is not regular is the language $\{0^n 1^n \mid n \in \mathbb{N}\}$

Since basic pushdown automata 'stack machine' capable of an arbitrary level of recursions. such a machine can cope with calculating Fibonacci(n) recursively for any large n , for

instance, without croaking. This a FMS cannot do because, with only a finite set of states (however large that set might be), it cannot have enough to cope with an *arbitrarily* large n if required to calculate Fibonacci(n) recursively - by definition. This holds true for all calculations involving recursion.

Also since, Finite state automata cannot parse context free languages that are not regular
 For instance, there is no FSA for solving the decision problem: In this particular case, the problem is the absence Given $w \in \{1,0\}^*$, Does $w \in L = \{0^n 1^n : n \geq 0\}$?

In this particular case, the problem is the absence
 Given $w \in \{1,0\}^*$, Does $w \in L = \{0^n 1^n : n \geq 0\}$? In this particular case, the problem is the absence in FSAs of a mechanism for keeping track of the occurrences of a symbol in a string. Such a problem needs an additional string which is a stack so, Pushdown automata solves this problem.

Shortages and Drawbacks:

It is known that the class of PDAs is not closed under intersection and complementation. Thus, decision problems like inclusion and equivalence are undecidable for PDAs.

Extra Knowledge:

Deterministic vs. nondeterministic PDA

A pushdown automaton is a deterministic pushdown automaton (DPDA) if

$$\forall q \in Q \wedge \forall a \in A \wedge \forall \gamma \in \Gamma \cup \{\lambda\} \wedge \forall \alpha \in \Gamma \cup \{\lambda\}$$

$\partial(q, a, \gamma, \alpha)$ is a non-empty singleton set

A pushdown automaton is a nondeterministic pushdown automaton (NPDA) if

$$\exists q \in Q \vee \exists a \in A \cup \{\lambda\} \vee \exists \gamma \in \Gamma \cup \{\lambda\} \vee \exists \alpha \in \Gamma \cup \{\lambda\}$$

$\partial(q, a, \gamma, \alpha)$ is a empty or not singleton set

Computation

A computation with input string $W = a_1 \dots a_k$ in a DPDA is a list of pairs in $Q \times \Gamma$, denoted

$$C(w) = \langle (q_0, \lambda), (q_1, \sigma_1), \dots, (q_k, \sigma_k) \rangle$$

Where :

$$(\forall i = 1, \dots, k) (\exists \gamma_i \in \Gamma \cup \{\lambda\} \vee \exists \alpha_i \in \Gamma \cup \{\lambda\})$$

$$\{(q_0, \sigma_i)\} = \partial (q_{i-1}, a_i, \gamma_i, \alpha_i)$$

With $\sigma_0 = \lambda$ and $\forall i = 1, \dots, k$ σ_i is the current top symbol in the stack at the particular stage in the computation

Accepting and Rejecting Computations

The previous computation is declared to be an accepting computation if its last pair (q_k, σ_k) satisfies $P(q_k, \sigma_k) : q_k \in F$

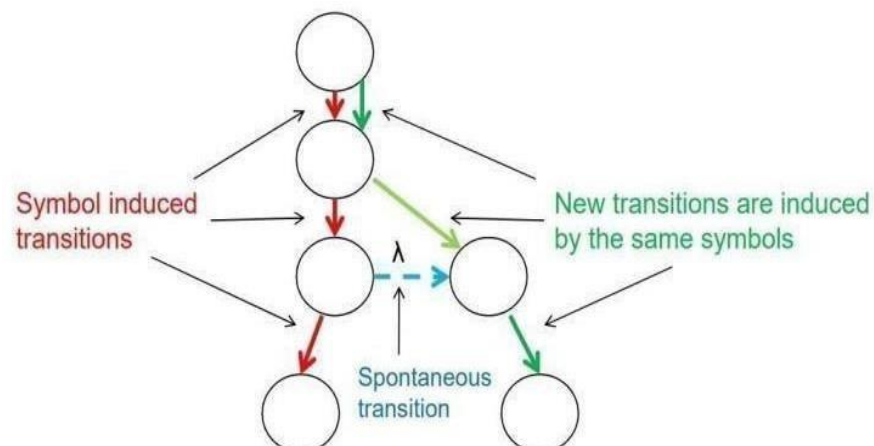
$$\wedge \sigma_k = \lambda$$

Conversely, the previous computation is a rejecting computation if its last pair satisfies instead

$$\neg P(q_k, \sigma_k) \Leftrightarrow q_k \notin F \wedge \sigma_k \neq \lambda$$

Nondeterministic spontaneous transitions

As with NFSA's, spontaneous transitions are modeled as the **spawn** of a **new thread in an existing thread** of computation. Graphically,



Nondeterministic Computations

The union of all computation threads generated either by the reading of W or created spontaneously as new computation threads during the reading constitutes the parsing or computation of a string w with a nondeterministic pushdown automaton.

A nondeterministic pushdown automaton accepts a string W if at least one accepting computation thread is finished with its computation.

Example

Let's illustrate the previous concepts by building a nondeterministic pushdown automaton for recognizing the context free language:

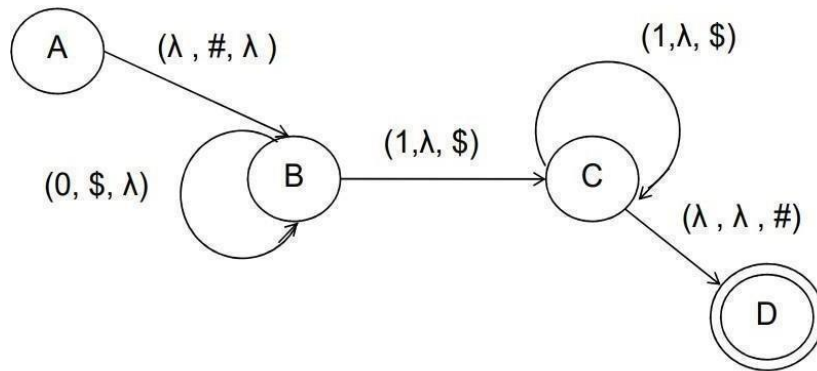
$$L = \{ 0^n 1^n : n \geq 0 \}$$

The automaton must have a mechanism for deciding whether there is the same number of 0's and 1's.

- This is done by pushing a symbol into the stack for each 0 read, and popping it out for each 1 read
- Define $P = (\{A, B, C, D\}, \{0, 1\}, \{\$, \#\}, t, A, \{D\})$, where t is defined by the table

t	$(\lambda, \#, \lambda)$	$(0, \$, \lambda)$	$(1, \lambda, \$)$	$(\lambda, \lambda, \#)$
A	{B}			
B		{B}	{C}	
C			{C}	{D}

As usual, undefined transitions are assumed to be empty transitions



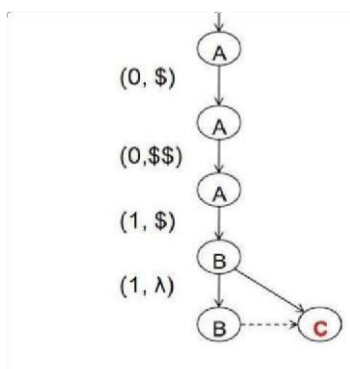
Threads of Configuration

A configuration is a triple in $A \times Q \times \Gamma^*$ where A and Γ are the alphabet and stack alphabet of a pushdown automaton, respectively.

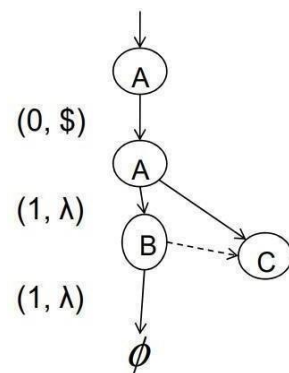
A thread of configurations is a sequence of configurations, denoted as

$(\lambda, q_0, \lambda) \rightarrow (a_1, q_1, s_1) \rightarrow \dots \rightarrow (a_k, q_k, s_k)$ which is used to convey all information in a computation thread

Represent Some Computations



String 0011



String 011

Turing Machine:

Definition:

___The turing machine is a mathematical model of computation which describes a machine that manipulates symbols on a strip of tape based on tabulated rules . This tape is divided into cells, each one of those cells is capable of holding a single symbol. however, it has the capability to execute any computing algorithm. Associated with the tape is a read-write head that can travel right or left on the tape and that can read and write a single symbol on each move. A Turing machine M is defined by

$M = (Q, \Sigma, \Gamma, \delta, q_0, F)$, where Q is

the set of internal states,

Σ is the input alphabet,

Γ is a finite set of symbols called the tape alphabet, δ is the transition function,

$\epsilon \in \Gamma$ is a special symbol called the blank, $q_0 \in Q$ is the initial state,

$F \subseteq Q$ is the set of final states.

Implementations:

-The Turing machine could be used as an example of a CPU that has the control of data manipulation done by a computer by using sequential memory to store data. The machine is capable of identifying some arbitrary subset of valid strings of an alphabet. In addition to this, the Turing machine has the capability of processing an unspecified grammar, which indicated that it can evaluate first-order logic in an unlimited amount of methods. A famous example of this is the lambda calculus. The Turing device can also mathematically model a machine that mechanically operates on a tape where there are symbols which the machine can read and write, at one time through using the tape head. Operation is determined by an infinite set of elementary instructions such as in state 36 if the symbol seen is 0, write a 1 else if the symbol seen is 1 the state would be modified into 18; in other words the Turing machine storage is simply one-dimensional array of cells, each of which can hold a single symbol. This array extends indefinitely in both directions and is therefore capable of holding an unlimited amount of information. We can visualize the Turing machine as a rather simple computer. It has a processing unit, which has a finite memory, and in its tape, it has a secondary storage of unlimited capacity. The instructions that such a computer can carry out are very limited: It can sense a symbol on its tape and use the result to decide what to do next. The only actions the machine can perform are to rewrite the current symbol, to change the state of the control, and to move the read-write head.

Advantages and disadvantages:

Disadvantages:

-Turing machines do not model the strengths of a particular arrangement reliably for instance, modern stored-program computers are actually examples of a more special form of abstract machine known as the random-access stored-program machine or abbreviation as RASP machine model

-Another disadvantage is that they do not model concurrency very well for example: there is a limit on the size of integer that can be calculated by an always halting nondeterministic Turing machine starting on a blank tape on the other hand there is

halting concurrent systems that can compute an integer of unbounded size

Advantages:

Turing devices have the advantage of unlimited memory in addition to this they are capable of simulating common computers. The Turing machine has the merit of being simple in a sense that the current machine state has only constant size all the information you need in order to determine the next machine state is one symbol and one control state number. Another advantage of Turing machine is that it has a tape head which can be propelled backward or forward, and the input tape can be scanned.

Problems it can solve:

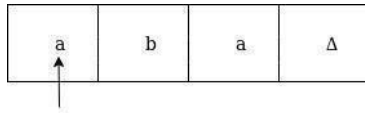
Turing machines are much more powerful than the automata machines. In fact, they solve precisely the set of all problems that can be solved by any digital computing device. Turing machines can do some simple addition, subtraction, multiplication and division operations, perform string manipulations, and make some simple comparisons. Turing machines could be used as Language

Accepters for example: A string w is written on the tape, with blanks filling out the unused portions. The machine is started in the initial state q_0 with the read-write head positioned on the leftmost symbol of w . If, after a sequence of moves, the Turing machine enters a final state and halts, then w is considered to be accepted.

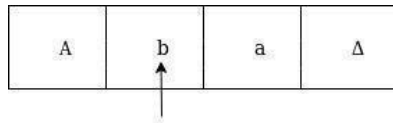
Let's take a look at the following example: let us consider a Turing machine which accepts the language of aba over $\Sigma = \{a, b\}$. If we suppose the string aba is placed like this:

a	b	a	Δ	-----
---	---	---	----------	-------

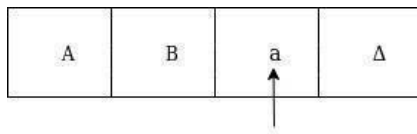
The tape head would read up to the 'delta' symbol , If the tape head is readout 'aba' string then Turing device will stop after reading the 'delta symbol'. Now, we will see how this turing tool will work for the string of aba. To begin with the , state is q_0 and head points to a as:



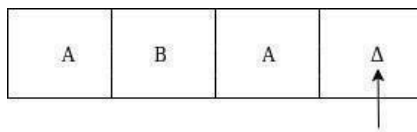
The move will be $\delta(q_0, a) = \delta(q_1, A, R)$ which means it will move to the state of q_1 , replaced a by A and head will traverse towards the right as:



The move will be $\delta(q_1, b) = \delta(q_2, B, R)$ which means it will traverse to state q_2 , replaced b by B and head will move to right as:



The move will be $\delta(q_2, a) = \delta(q_3, A, R)$ which means it will go to state q_3 , replaced a by A and head will change its position towards right as:



The move $\delta(q_3, \Delta) = (q_4, \Delta, S)$ which means it will go to state q_4 which is the HALT state and HALT state is always an accepted state for any kind of Turing Machine.

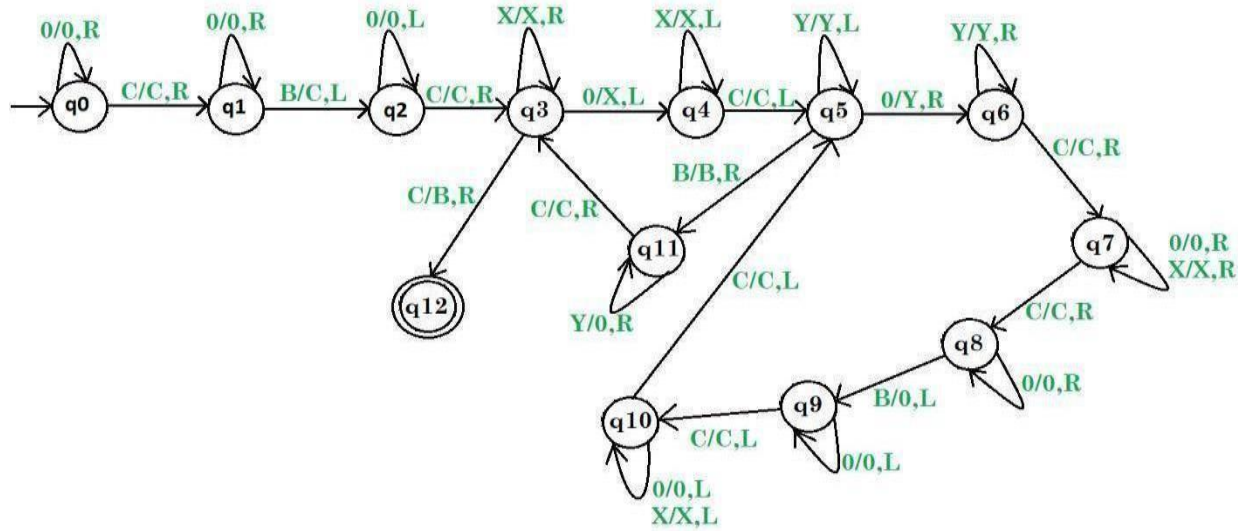


Figure 1 turing machine for multiplaction

Extra Knowledge:

History:

The turing machine was invented by the famous English mathematician in 1936 by Alan turing. Turing is considered to be the father of theortical computer science and artificial intelligence . Alan Turing described what became known as the "Turing Machine" in his 1936 document , "On Computable Numbers, with an application to the

Entscheidungsproblem," which was published in the Proceedings of the London Mathematical Society back in the late 1930s . The machine was not a piece of equipment but a concept ,however , its principles set the way for the development of digital computers in the 20th century. As mentioned previously all problems can be solved by digital computers can be solved by turing machine, however, there are some problems that are unsolvable by the turing machine such as the halting problem which is defined as : Given an arbitrary Turing machine M over alphabet $= \{ a , b \}$, and an arbitrary string w over , does M halt when it is given w as an input ?

Figure 2 halt python cod

```

Python
1  x = input()
2  while x:
3      pass

```

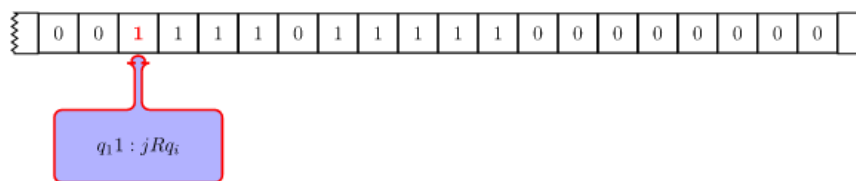
Computing with Turing Machines:

Turing machines were intended to formalize the notion of computability in order to solve fundamental mathematical problems. Alonzo Church gave a different but logically equal formulation. Today, most computer scientists agree that

Turing's, or any other logically equal, formal notion captures all computable problems, viz. for any computable problem, there is a Turing machine which can compute it. It indicates that, if accepted, any problem not computable by a Turing device, it would be impossible to be computed using any finite method. Let's consider the following example: we will represent the number n as a block of $n+1$ copies of the symbol '1' on the tape. Thus we will represent the number

0 as a single '1' and the number 3 in the form of block of four '1's. This method is called as unary notation. We will also have to produce some assumptions about the configuration of the tape when the machine is started, and when it ends, in order to interpret the computation. We will assume that if the function to be computed requires n arguments, then the Turing machine will start with its head scanning the leftmost '1' of a sequence of n blocks of '1's. The blocks of '1's representing the arguments must be separated by a single occurrence of the symbol '0'. For example, to compute the sum $3+4$, a Turing machine will start in the configuration shown in Figure 3 :

Figure 3 Initial configuration for a computation over two numbers n and



The turing machine takes two arguments which symbolize the numbers to be added, beginning at the leftmost 1 of the first argument. The arguments are separated from each other by a single 0 as needed, and the first block contains four '1's, representing the number 3, and the second contains five '1's, representing the number 4. A machine must finish in standard configuration too. There must be a single block of symbols (a sequence of 1s which symbolize some number or a symbol representing another kind of output) and the machine must be scanning the leftmost symbol of that sequence. If the machine computes the function correctly then this block must represent the correct output.

	0	1
q_1	/	0 R q_2
q_2	1 L q_3	1 R q_2
q_3	0 R q_4	1 L q_3
q_4	/	0 R q_{halt}

Table 1 Transition table for turing machine of adding 2 natural numbers

Table 1 displays the transition table of a Turing machine called TAdd2 which performs addition on 2 natural numbers n and m . We suppose that the machine starts in state q_1 scanning the leftmost 1 of $n+1$. The idea of doing an addition with Turing machines when using unary representation is to shift the leftmost number n one square to the right. This is achieved by deleting the leftmost 1 of $n+1$ (this is done in state q_1) and then setting the 0 between $n+1$ and $m+1$ to 1 (state q_2). We then have $n+m+2$ and so we still need to erase one additional 1. This is done by erasing the leftmost 1 (states q_3 and q_4). Figure 4 shows this computation for $3+4$

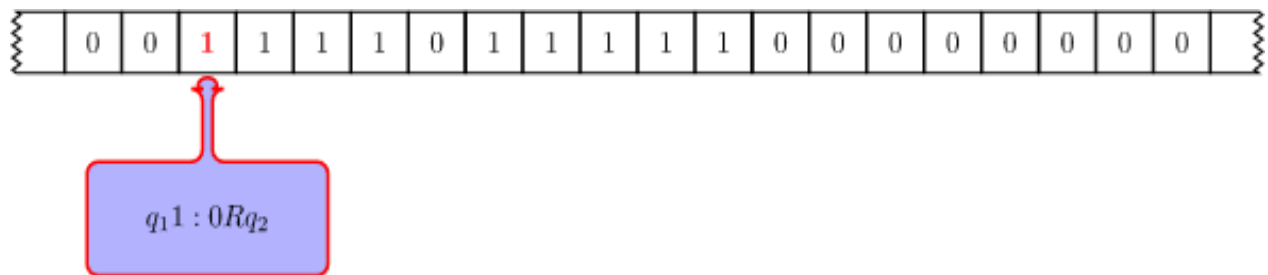


Figure 4

Examples on CFG to PDA:

CFG for Balanced Parenthesis:

Context-Free Grammar Input

Grammar Rules

```
S -> (S)
S -> SS
S -> epsilon
```

Format: A -> alpha (one rule per line)
Use 'epsilon' for ϵ -productions
Comments start with #

OUTPUT:

Pushdown Automaton Output

```
Pushdown Automaton (PDA):
=====

States (Q): [q1, q2, q0]
Input Alphabet (Sigma): []
Stack Alphabet (Gamma): [0, S, Z0, Z]
Initial State (q0): q0
Initial Stack Symbol (Z0): Z0
Accepting States (F): [q2]

Transition Function (delta):
-----
delta(q0, epsilon, Z0) = (q1, SZ0)
delta(q1, epsilon, S) = (q1, epsilon)
delta(q1, epsilon, Z0) = (q2, epsilon)
```

Pushdown Automaton Output

CFG to PDA Conversion Process:

=====

1. Create three states: q0 (initial), q1 (loop), q2 (final)
2. Set initial stack symbol: Z0
3. Create initial transition: (q0, epsilon, Z0) -> (q1, SZ0)
4. For each production rule, create epsilon transition:
Rule: S -> epsilon
Transition: (q1, epsilon, S) -> (q1, epsilon)
5. For each terminal symbol, create matching transition:
6. Create final transition: (q1, epsilon, Z0) -> (q2, epsilon)

Grammar Complexity Analysis:

=====

Basic Metrics:

- Non-terminals: 1
- Terminals: 0
- Production rules: 1

Recursion Analysis:

- Left recursive rules: 0
- Right recursive rules: 0
- Self-recursive non-terminals: 0

Complexity Assessment:

- Overall complexity: LOW
- Expected PDA size: Small (< 20 transitions)

CFG for Palindromes over {a, b}:

 CFG to PDA Converter

File Edit Examples Help

Context-Free Grammar Input

Grammar Rules

```
S -> (S)
S -> aSa
S -> (S)
S -> aSaSaa
S -> bS
S -> b
S -> epsilon
```

Valid Grammar



Grammar validation successful!

OK

Format: A -> alpha (one rule per line)

Use 'epsilon' for ϵ -productions

Comments start with #

Validation Messages

Grammar is valid!

Lines: 7, Rules: 7

Ready

OUTPUT:

Pushdown Automaton Output

Pushdown Automaton (PDA):

=====

States (Q): [q1, q2, q0]

Input Alphabet (Sigma): [b]

Stack Alphabet (Gamma): [0, b, S, Z0, Z]

Initial State (q0): q0

Initial Stack Symbol (Z0): Z0

Accepting States (F): [q2]

Transition Function (delta):

delta(q0, epsilon, Z0) = (q1, SZ0)

delta(q1, epsilon, S) = (q1, b)

delta(q1, epsilon, S) = (q1, epsilon)

delta(q1, b, b) = (q1, epsilon)

delta(q1, epsilon, Z0) = (q2, epsilon)

CFG to PDA Conversion Process:

=====

1. Create three states: q0 (initial), q1 (loop), q2 (final)
2. Set initial stack symbol: Z0
3. Create initial transition: (q0, epsilon, Z0) -> (q1, SZ0)
4. For each production rule, create epsilon transition:
Rule: S -> b
Transition: (q1, epsilon, S) -> (q1, b)

Rule: S -> epsilon
Transition: (q1, epsilon, S) -> (q1, epsilon)
5. For each terminal symbol, create matching transition:
Terminal: b
Transition: (q1, b, b) -> (q1, epsilon)
6. Create final transition: (q1, epsilon, Z0) -> (q2, epsilon)

Grammar Complexity Analysis:

Basic Metrics:

- Non-terminals: 1
- Terminals: 1
- Production rules: 2

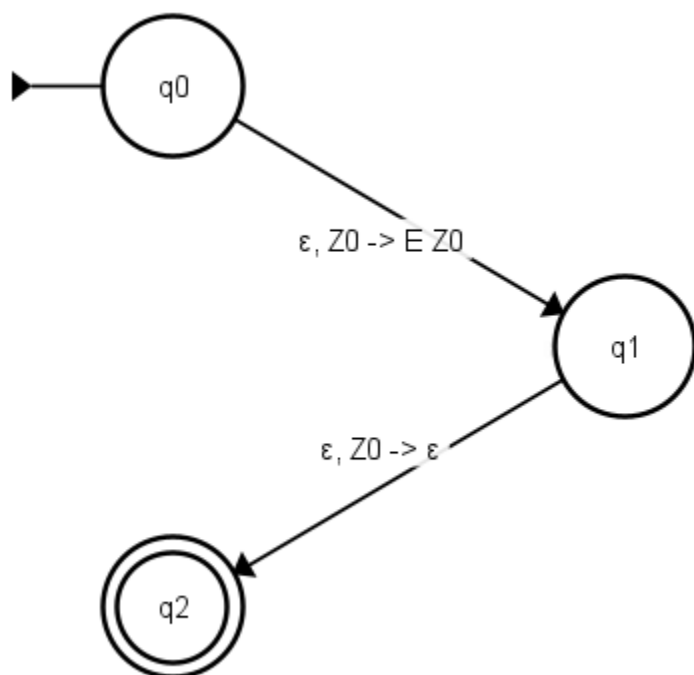
Recursion Analysis:

- Left recursive rules: 0
- Right recursive rules: 0
- Self-recursive non-terminals: 0

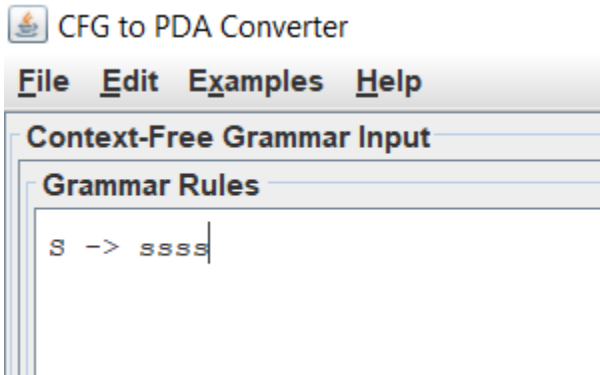
Complexity Assessment:

- Overall complexity: LOW
- Expected PDA size: Small (< 20 transitions)

Recommendations:



Cases with Error Handling :

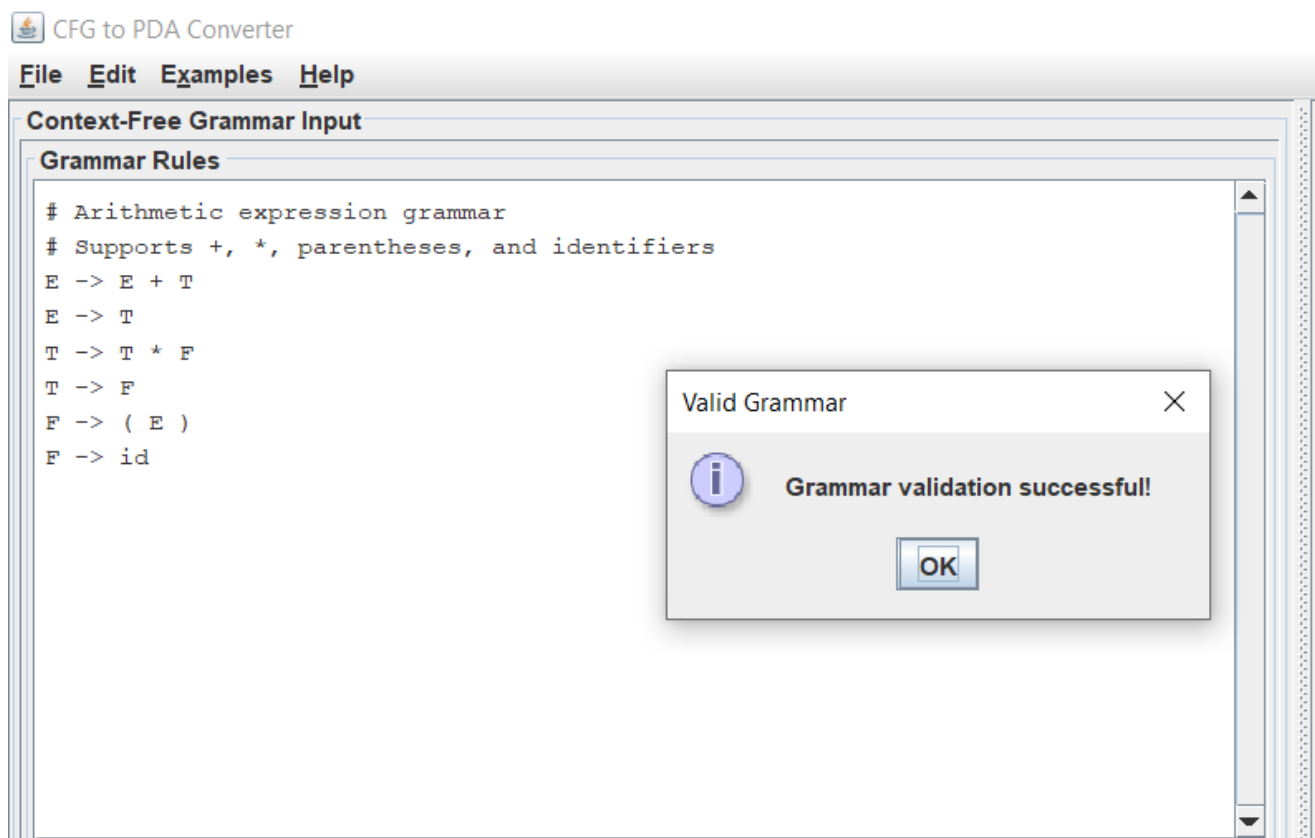


This will output the following:

Line 1: Invalid symbol 'ssss'. Symbols must be uppercase letters (non-terminals), lowercase letters/digits (terminals), or special symbols (+, *, (,), id, epsilon)
Grammar must contain at least one production rule
Grammar must have a start symbol

Arithmetic Grammar examples:

Input:



Output:

Pushdown Automaton Output

Pushdown Automaton (PDA):

=====

States (Q): [q1, q2, q0]

Input Alphabet (Sigma): [(,), *, +, id]

Stack Alphabet (Gamma): [d, E, F, (,), i, *, +, 0, T, Z0, Z, id]

Initial State (q0): q0

Initial Stack Symbol (Z0): Z0

Accepting States (F): [q2]

Transition Function (delta):

delta(q0, epsilon, Z0) = (q1, EZ0)

delta(q1, epsilon, E) = (q1, T+E)

delta(q1, epsilon, E) = (q1, T)

delta(q1, epsilon, T) = (q1, F*T)

delta(q1, epsilon, T) = (q1, F)

delta(q1, epsilon, F) = (q1,)E()

delta(q1, epsilon, F) = (q1, id)

delta(q1, (, () = (q1, epsilon)

delta(q1,),)) = (q1, epsilon)

delta(q1, *, *) = (q1, epsilon)

delta(q1, +, +) = (q1, epsilon)

delta(q1, id, id) = (q1, epsilon)

delta(q1, epsilon, Z0) = (q2, epsilon)

CFG to PDA Conversion Process:

1. Create three states: q_0 (initial), q_1 (loop), q_2 (final)
2. Set initial stack symbol: Z_0
3. Create initial transition: $(q_0, \text{epsilon}, Z_0) \rightarrow (q_1, EZ_0)$
4. For each production rule, create epsilon transition:
Rule: $E \rightarrow E + T$
Transition: $(q_1, \text{epsilon}, E) \rightarrow (q_1, T+E)$

Rule: $E \rightarrow T$
Transition: $(q_1, \text{epsilon}, E) \rightarrow (q_1, T)$

Rule: $T \rightarrow T * F$
Transition: $(q_1, \text{epsilon}, T) \rightarrow (q_1, F*T)$

Rule: $T \rightarrow F$
Transition: $(q_1, \text{epsilon}, T) \rightarrow (q_1, F)$

Rule: $F \rightarrow (E)$
Transition: $(q_1, \text{epsilon}, F) \rightarrow (q_1,)E()$

Rule: $F \rightarrow \text{id}$
Transition: $(q_1, \text{epsilon}, F) \rightarrow (q_1, \text{id})$
5. For each terminal symbol, create matching transition:
Terminal: $($
Transition: $(q_1, (, () \rightarrow (q_1, \text{epsilon})$
Terminal: $)$
Transition: $(q_1,),)) \rightarrow (q_1, \text{epsilon})$
Terminal: $*$
Transition: $(q_1, *, *) \rightarrow (q_1, \text{epsilon})$
Terminal: $+$
Transition: $(q_1, +, +) \rightarrow (q_1, \text{epsilon})$
Terminal: id
Transition: $(q_1, \text{id}, \text{id}) \rightarrow (q_1, \text{epsilon})$
6. Create final transition: $(q_1, \text{epsilon}, Z_0) \rightarrow (q_2, \text{epsilon})$

Grammar Complexity Analysis:

=====

Basic Metrics:

- Non-terminals: 3
- Terminals: 5
- Production rules: 6

Recursion Analysis:

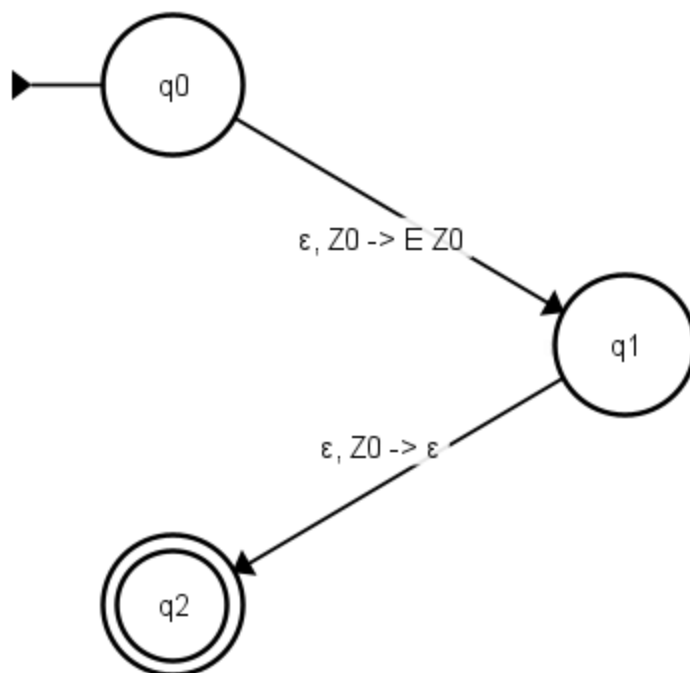
- Left recursive rules: 2
- Right recursive rules: 0
- Self-recursive non-terminals: 2

Complexity Assessment:

- Overall complexity: MEDIUM
- Expected PDA size: Medium (20-50 transitions)

Recommendations:

- Consider eliminating left recursion for better parsing performance
- High recursion detected - ensure adequate stack space for PDA simulation



References:

<https://www.cs.odu.edu/~zeil/cs390/latest/Public/pda-iflap/index.html>

https://en.wikipedia.org/wiki/Pushdown_automaton

<https://cstheory.stackexchange.com/questions/9673/why-is-non-determinism-push-down-automata-necessary>

<https://www.geeksforgeeks.org/introduction-of-pushdownautomata/>

<https://www.javatpoint.com/automata-turing-machine> https://en.wikipedia.org/wiki/Turing_machine

<https://plato.stanford.edu/entries/turingmachine/#CompTuriMach>

<https://www.slideshare.net/HimanshuSirohi6/turing-machine-132443033>